# Linux File Permissions and Ownership Notes

## Understanding File Permissions

### 1. File Permissions Structure

File permissions are displayed in a 10-character format when using `ls -l`:

```
$ ls -l Desktop/
drwxr-xr-x 2 pete penguins 4096 Dec 1 11:45 .
```

#### Permission Structure Breakdown:

```
d | rwx | r-x | r-x
^   ^     ^     ^
|   |     |     └── Other permissions
|   |     └──────── Group permissions
|   └────────────── User permissions
└──────────────────── File type
```

#### File Type Indicators:

- `d` = directory
- `-` = regular file
- `l` = symbolic link
- `c` = character device
- `b` = block device

#### Permission Characters:

- `r` = readable (4)
- `w` = writable (2)
- `x` = executable (1)
- `-` = no permission (0)

#### Example Analysis:

```
drwxr-xr-x 2 pete penguins 4096 Dec 1 11:45 .
```

- File type: `d` (directory)
- User (pete): `rwx` (read, write, execute)
- Group (penguins): `r-x` (read, execute only)
- Others: `r-x` (read, execute only)

# Modifying Permissions

## 2. chmod Command - Symbolic Method

### Adding permissions:

```
$ chmod u+x myfile
# Adds execute permission for user

$ chmod g+w myfile
# Adds write permission for group

$ chmod o+r myfile
# Adds read permission for others

$ chmod ug+w myfile
# Adds write permission for user and group
```

### Removing permissions:

```
$ chmod u-x myfile
# Removes execute permission from user

$ chmod g-w myfile
# Removes write permission from group

$ chmod o-r myfile
# Removes read permission from others
```

### Setting exact permissions:

```
$ chmod u=rwx,g=rx,o=r myfile
# Sets user to rwx, group to rx, others to r
```

## 3. chmod Command - Numerical Method

### Permission Values:

- Read (r) = 4
- Write (w) = 2
- Execute (x) = 1

### Common Permission Combinations:

```
$ chmod 755 myfile
# 7 = 4+2+1 (rwx) for user
# 5 = 4+1 (r-x) for group
# 5 = 4+1 (r-x) for others
# Result: rwxr-xr-x

$ chmod 644 myfile
# 6 = 4+2 (rw-) for user
# 4 = 4 (r--) for group
# 4 = 4 (r--) for others
# Result: rw-r--r--

$ chmod 777 myfile
# 7 = 4+2+1 (rwx) for all
# Result: rwxrwxrwx (full permissions)

$ chmod 000 myfile
# No permissions for anyone
# Result: ---------
```

# Ownership Management

## 4. Changing File Ownership

### Change user ownership:

```
$ sudo chown patty myfile
# Changes owner to 'patty'

$ ls -l myfile
-rw-r--r-- 1 patty currentgroup 1234 Dec 1 12:00 myfile
```

### Change group ownership:

```
$ sudo chgrp whales myfile
# Changes group to 'whales'
```

```
$ ls -l myfile
-rw-r--r-- 1 currentuser whales 1234 Dec 1 12:00 myfile
```

### Change both user and group:

```
$ sudo chown patty:whales myfile
# Changes owner to 'patty' and group to 'whales'

$ ls -l myfile
-rw-r--r-- 1 patty whales 1234 Dec 1 12:00 myfile
```

### Recursive ownership change:

```
$ sudo chown -R patty:whales mydirectory/
# Changes ownership of directory and all contents
```

# Default Permissions

## 5. umask - Default Permission Mask

The `umask` command sets default permissions for newly created files by **removing** permissions:

```
$ umask
022
# Current umask value

$ umask 021
# Sets new umask: remove write for group, execute for others
```

### Understanding umask values:

- `umask 022`: Remove write permission for group and others
  - Files created with: 644 (rw-r--r--)
  - Directories created with: 755 (rwxr-xr-x)
- `umask 077`: Remove all permissions for group and others
  - Files created with: 600 (rw-------)
  - Directories created with: 700 (rwx------)

### Example:

```
$ umask 002
$ touch newfile.txt
```

```
$ ls -l newfile.txt
-rw-rw-r-- 1 user group 0 Dec 1 12:00 newfile.txt
# Group has write permission, others have read only
```

# Special Permission Bits

## 6. SUID (Set User ID)

SUID allows a program to run with the permissions of the file owner:

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 47032 Dec 1 11:45 /usr/bin/passwd
#    ^
#    └── SUID bit (s instead of x)
```

### Why SUID is needed:

```
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1134 Dec 1 11:45 /etc/shadow
# Only root can write to this file

$ passwd
# But users can change passwords because passwd has SUID
# When running passwd, you temporarily become root
```

### Setting SUID:

```
# Symbolic method
$ sudo chmod u+s myfile

# Numerical method
$ sudo chmod 4755 myfile
# 4 = SUID bit, 755 = regular permissions
# Result: rwsr-xr-x
```

### SUID with capital S:

```
-rwSr-xr-x 1 root root 1234 Dec 1 12:00 myfile
#   ^
#   └── Capital S means SUID is set but no execute permission
```

## 7. SGID (Set Group ID)

SGID allows a program to run with the permissions of the file's group:

```
$ ls -l /usr/bin/wall
-rwxr-sr-x 1 root tty 19024 Dec 14 11:45 /usr/bin/wall
#      ^
#      └── SGID bit (s in group position)
```

### Setting SGID:

```
# Symbolic method
$ sudo chmod g+s myfile

# Numerical method
$ sudo chmod 2755 myfile
# 2 = SGID bit, 755 = regular permissions
# Result: rwxr-sr-x
```

### SGID on directories:

```
$ sudo chmod g+s mydir
$ ls -ld mydir
drwxrwsr-x 2 user group 4096 Dec 1 12:00 mydir
# Files created in this directory inherit the group ownership
```

## 8. Sticky Bit

The sticky bit prevents users from deleting files they don't own:

```
$ ls -ld /tmp
drwxrwxrwt 6 root root 4096 Dec 15 11:45 /tmp
#        ^
#        └── Sticky bit (t at the end)
```

### How sticky bit works:

- Everyone can create files in `/tmp`
- Everyone can modify their own files
- Only file owners (or root) can delete files
- Perfect for shared directories

### Setting sticky bit:

```
# Symbolic method
$ sudo chmod +t mydir

# Numerical method
$ sudo chmod 1755 mydir
# 1 = sticky bit, 755 = regular permissions
# Result: rwxr-xr-t
```

# Process Permissions

## 9. Understanding UIDs in Processes

Every process has three types of UIDs:

### Real UID (RUID):

- The actual user who launched the process
- Used for tracking who started the process

### Effective UID (EUID):

- The UID used for permission checks
- Usually same as real UID, but changes with SUID

### Saved UID (SUID):

- Allows switching between real and effective UID
- Provides security by limiting privilege escalation

### Example with passwd command:

```
# Normal user (UID 500) runs passwd
$ passwd

# Process starts with:
# Real UID: 500 (your user ID)
# Effective UID: 0 (root, because of SUID bit)
# Saved UID: 500 (your original ID)

# You can change YOUR password (UID 500)
# But you CANNOT change Sally's password (UID 600)
# Because your Real UID is still 500
```

# Special Permission Combinations

## 10. Combining Special Bits

You can combine multiple special permission bits:

```
$ sudo chmod 6755 myfile
# 6 = 4 (SUID) + 2 (SGID)
# Result: rwsr-sr-x

$ sudo chmod 7755 myfile
# 7 = 4 (SUID) + 2 (SGID) + 1 (Sticky)
# Result: rwsr-sr-t
```

# Common Permission Patterns

## 11. Typical Permission Scenarios

```
# Executable programs
$ chmod 755 script.sh
# rwxr-xr-x - owner can modify, everyone can execute

# Configuration files
$ chmod 644 config.txt
# rw-r--r-- - owner can modify, others can read

# Private files
$ chmod 600 private.txt
# rw------- - only owner can read/write

# Directories
$ chmod 755 mydir/
# rwxr-xr-x - owner full access, others can enter and list

# Shared directories
$ chmod 1777 shared/
# rwxrwxrwt - everyone can create files, only owners can delete
```

## 12. Security Best Practices

```
# Check for SUID/SGID files (potential security risks)
$ find /usr -perm -4000 -o -perm -2000 2>/dev/null
```

```
# Check for world-writable files
$ find /home -perm -002 2>/dev/null

# Check for files with no owner
$ find /home -nouser -o -nogroup 2>/dev/null
```

# Quick Reference

## Permission Calculation Chart

| Permission | Binary | Decimal |
|---|---|---|
| --- | 000 | 0 |
| --x | 001 | 1 |
| -w- | 010 | 2 |
| -wx | 011 | 3 |
| r-- | 100 | 4 |
| r-x | 101 | 5 |
| rw- | 110 | 6 |
| rwx | 111 | 7 |

## Special Permission Bits Reference

| Bit | Symbol | Numeric | Description |
|---|---|---|---|
| SUID | s/S | 4 | Run as file owner |
| SGID | s/S | 2 | Run as file group |
| Sticky | t/T | 1 | Restrict deletion |